# Przegląd symulatorów ryby-robota

Sześć narzędzi symulacyjnych zastosowanych do jednej miękkiej, hydraulicznie napędzanej ryby-robota: co każde przyjmuje na wejściu, co zwraca, jak działa w środku i na jakie pytania potrafi odpowiedzieć.

Gałęzie: main · stonefish · pyelastica · sofa · openfoam · openmodelica  Wersja: 9 października 2026

Język: PL (dostępna wersja EN)

**Liczby pokazują mechanizmy, a nie ranking.** Każdy parametr fizyczny w sześciu demach jest wartością zastępczą (placeholder), niezidentyfikowaną na prawdziwym robocie. Dema używają też różnych zestawów parametrów (np. komora w SOFA ma 91 ml, w MuJoCo 30 ml). Prędkość 22 cm/s w jednym narzędziu i 44 cm/s w innym mówi, jaką fizykę każde z nich uwzględnia, a nie które jest dokładniejsze.

## 1 · Podsumowanie

Każde narzędzie odpowiada na inne pytanie o tę samą rybę. Trzy symulują całą rybę pływającą swobodnie, dwa przyglądają się z bliska miękkiemu ogonowi, a jedno modeluje cały napęd od baterii do płetwy.

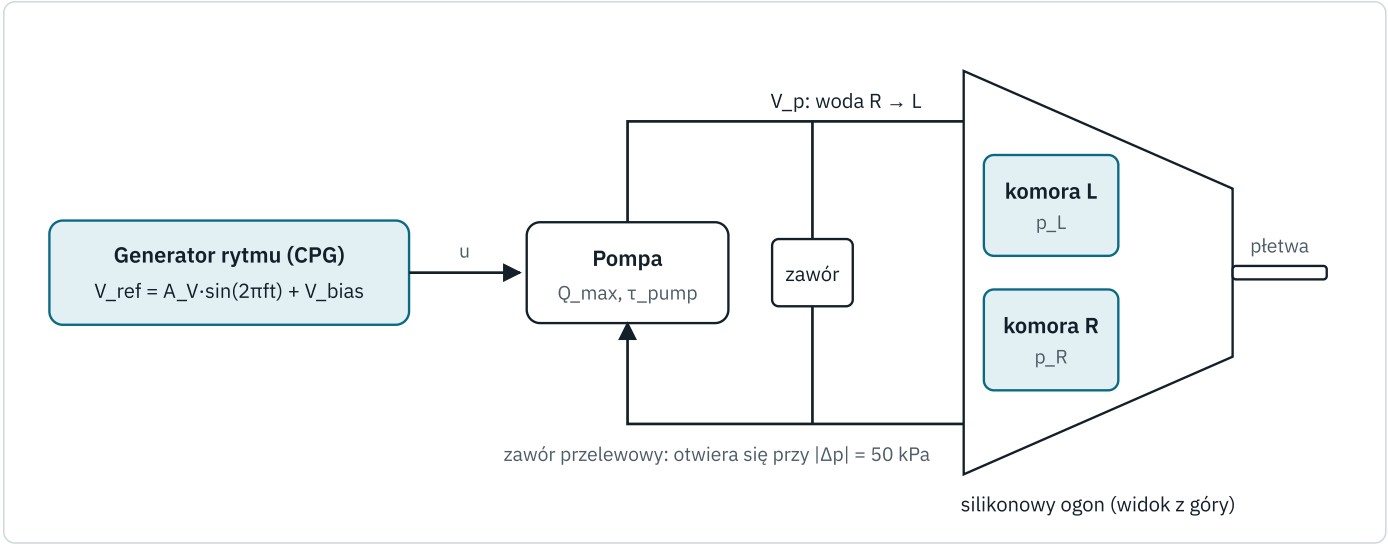
| NARZĘDZIE | GAŁĄŹ / KATALOG | POZIOM SZCZEGÓŁU | CO POKAZUJE W TYM REPO |
|---|---|---|---|
| **MuJoCo** | `main` · `MuJoCo/` | Cała ryba, sztywne segmenty | Pływanie, skręcanie, regulacja głębokości pęcherzem; sterowanie z klawiatury na żywo |
| **Stonefish** | `stonefish` · `Stonefish/` | Cała ryba, sztywne segmenty, środowisko morskie | Hydrodynamika z siatki, balast VBS, zaszumione czujniki ciśnienia i IMU, prąd wody |
| **PyElastica** | `pyelastica` · `fish_elastica_demo/` | Cała ryba jako jeden miękki pręt | Zginanie napędzane komorami, swobodne pływanie 3D, wpływ sztywności na prędkość, głębokość |
| **SOFA + SoftRobots** | `sofa` · `SOFA/` | Tylko ogon, elementy skończone 3D | Jak odkształca się silikonowy ogon i jego komory: krzywa p–V, kąt zgięcia, machanie w powietrzu i w wodzie |
| **OpenFOAM + preCICE + CalculiX** | `openfoam` · `OpenFOAM/` | Ogon i przepływ wody, 2D | Dwukierunkowa interakcja płyn–struktura: ślad wirowy, masa dodana, ciąg na uwięzi |
| **OpenModelica** | `openmodelica` · `FishRobot/` | Model systemowy, o parametrach skupionych | Bateria → silnik → pompa → przewody → komory → ogon; bilans energii, balast, eksport FMU, kalibracja |

### Które narzędzie do którego pytania

| PYTANIE | PIERWSZY WYBÓR | TEŻ PRZYDATNE |
|---|---|---|
| Czy ryba płynie, skręca i trzyma głębokość? Chcę stroić regulatory interaktywnie. | MuJoCo | Stonefish |
| Czy oprogramowanie sterujące działa z zaszumionymi czujnikami i prądem wody? Droga do ROS 2. | Stonefish | MuJoCo |
| Jak sztywność ciała i ogona zmienia prędkość, tanio i z ciągłym ciałem? | PyElastica | SOFA |
| Jakiego ciśnienia potrzebuje komora dla danej objętości i zgięcia? Która ścianka, włókna czy kręgosłup zgina najlepiej? | SOFA | OpenFOAM (część CalculiX) |
| Co naprawdę robi woda: ślad wirowy, masa dodana, rzeczywisty ciąg lub opór? | OpenFOAM FSI | żadne inne nie liczy przepływu |
| Czy silnik, pompa i bateria są dobrze dobrane? Na ile starczy baterii? Gdzie ginie energia? | OpenModelica | żadne inne nie ma elektryki |
| Jak zamienić pomiary ze stanowiska na parametry modelu? | OpenModelica (`calibrate.py`) | plany kalibracji w README MuJoCo i SOFA |

## 2 · Wspólny model ryby

Wszystkie sześć dem modeluje tę samą koncepcję robota: rybę z miękkim silikonowym ogonem zginanym przez wodę przepompowywaną między dwiema komorami. Znajomość tej pętli ułatwia czytanie tabel wejść w rozdziale 3.




Wspólna pętla hydrauliczna. Pompa przetłacza objętość V_p z komory R do komory L w obiegu zamkniętym; różnica ciśnień Δp = p_L − p_R zgina ogon. Generator rytmu zadaje objętość (a nie przepływ), a przesunięcie V_bias powoduje skręt. Głębokość jest regulowana osobno: PID ustawia objętość pęcherza balastowego (MuJoCo, PyElastica), zbiornika VBS (Stonefish) lub strzykawki (OpenModelica).

### Te same wielkości, inaczej ustawione w każdym demie

| WIELKOŚĆ | MUJOCO | STONEFISH | PYELASTICA | SOFA | OPENFOAM | OPENMODELICA |
|---|---|---|---|---|---|---|
| **Ciało** | kadłub-elipsoida 25×10×10 cm | jak w MuJoCo | cała ryba = pręt 0,40 m | brak (ogon utwierdzony) | głowa 2D, pół-elipsa 60 mm, nieruchoma | masa skupiona 1 kg |
| **Ogon** | 5 sztywnych segmentów × 40 mm, 3 napędzane | 5 sztywnych segmentów × 40 mm | ciągły, strefa komór 0,20–0,32 m | MES 3D, 0,20 m | MES 2D, 0,15 m | 1 obrotowy stopień swobody |
| **Wydajność pompy Q_max** | 60 ml/s | 60 ml/s | 160 ml/s | 250 ml/s | brak (zadane ciśnienie) | 0,3 ml na obrót |
| **Amplituda objętości A_V** | 8 ml | 8 ml | 8 ml | 17 ml | P0 = 15 kPa | wypełnienie CPG A = 0,8 |
| **Ciśnienie otwarcia zaworu** | 50 kPa | 50 kPa | 50 kPa | 50 kPa | brak | 50 kPa |
| **Balast** | pęcherz 0–40 ml | VBS 0,5–40,5 ml | pęcherz 0–40 ml | brak | brak | strzykawka 12,5 ml |

## 3 · Sześć narzędzi

Każdy rozdział narzędzia ma te same części: kartę faktów, zasadę działania, dane wejściowe (używane w demie i możliwe w narzędziu), dane wyjściowe z przykładowymi wartościami z repo, etapy dema i ograniczenia. Zrzuty ekranu i filmy pochodzą z samych programów, nagranych na potrzeby raportu 9 października 2026; gdy narzędzie nie ma GUI, pokazana jest animacja z dema.

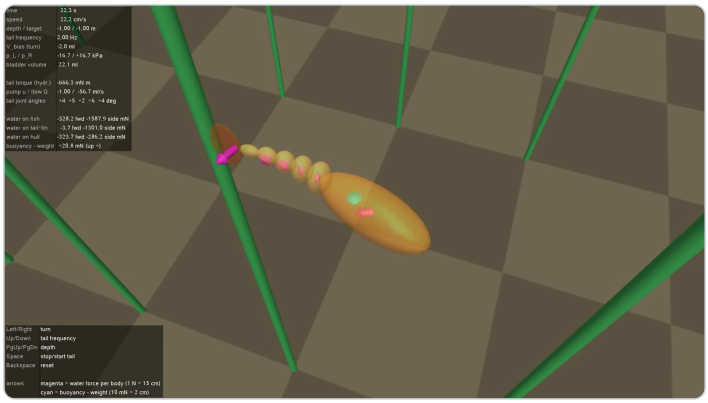
CAŁA RYBA · GAŁĄŹ MAIN

### MuJoCo

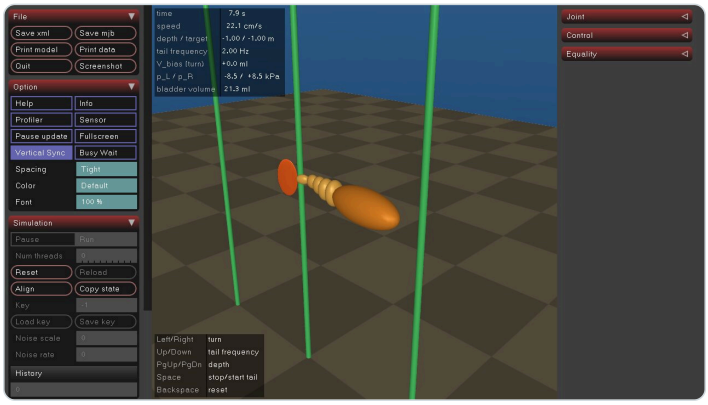
Szybki silnik dynamiki brył sztywnych z prostym wbudowanym modelem wody. Ryba to kadłub i łańcuch pięciu sztywnych segmentów ogona; wyporność i hydraulika są dodane w Pythonie.

| METODA | INTERFEJS | TESTOWANA WERSJA |
|---|---|---|
| Dynamika układu brył sztywnych + quasi-statyczne siły płynu na elipsoidach | Wiązania Pythona, model MJCF generowany w kodzie | MuJoCo 3.15.0, Python ≥ 3.10 |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| `pip install -r requirements.txt`, kilka minut | Na żywo w interaktywnym podglądzie; scenariusze 10–130 s bez GUI | Apache-2.0 |

## W programie: GUI i film demonstracyjny



**FILM** Podgląd MuJoCo nagrany na żywo w 2560×1440 i spowolniony do 0,5× (30 s symulacji w 61 s). Nakładka pokazuje stan (czas, prędkość, głębokość, częstotliwość ogona, V_bias, ciśnienia w komorach, objętość pęcherza), napęd (moment hydrauliki na ogonie, sterowanie i przepływ pompy, kąty przegubów ogona) oraz siły wody na całą rybę, na ogon z płetwą i na kadłub. Siły są rozłożone na składową naprzód i w bok, w mN. Strzałki magenta to siła wody na każde ciało, cyjanowa to wypór – ciężar. Ciała są przezroczyste, żeby strzałki było widać. MuJoCo podaje siłę płynu tylko łącznie, więc rozkład na ciała jest liczony ponownie na kopii stanu (`fishsim/readouts.py`). Skrypt naciska klawisze w tych chwilach symulacji: → w 8 s, ← w 17 s, → w 26 s, PgDn w 28 s (zanurzenie). Plik filmu: `docs/media/mujoco_demo.mp4` (odtwarzany w wersji HTML).




**EKRAN** Pełne okno podglądu MuJoCo: panele File, Option i Simulation po lewej, Joint, Control i Equality po prawej, nakładka stanu dema na scenie.

## Zasada działania

- **Ciało.** Kadłub to elipsoida na przegubie swobodnym. Ogon to pięć elipsoid na zawiasach ze sprężynami i tłumikami, czyli przybliżenie silikonu modelem pseudo-sztywnym. Płetwa ogonowa to płaska elipsoida na ostatnim segmencie.
- **Woda.** Wbudowany model elipsoidalny MuJoCo daje opór czołowy i boczny, opór obrotowy, siłę nośną Kutty i siłę Magnusa dla każdej części. Nie ma pola przepływu.
- **Masa dodana, poprawiona.** MuJoCo stosuje tylko żyroskopową część siły masy dodanej: kod źródłowy (`engine_passive.c`) pomija człon z przyspieszeniem, który pokazuje dokumentacja. Sama ta część pchała rybę do tyłu (4,5 cm/s zamiast 22 cm/s). Demo ją wyłącza i dodaje masę dodaną jako `armature` przegubów.
- **Wyporność.** MuJoCo nie liczy wyporu. Demo przykłada go do każdej bryły, ze środkiem wyporu stałym w układzie kadłuba, więc przechył daje moment prostujący. Masa kadłuba jest wyliczana z neutralnej pływalności przy pęcherzu wypełnionym do połowy.
- **Hydraulika.** Model w Pythonie działa przed każdym krokiem fizyki: pompa pierwszego rzędu, $\Delta p = (V_p - A \cdot r \cdot L)/C_h$ obcięte do ±50 kPa i siła cięgna $F = A \cdot r \cdot \Delta p$ rozłożona na trzy napędzane przeguby z wagami od 1 do 0,4.
- **Sterowanie.** Objętościowy generator rytmu dla ogona i kaskadowy PID głębokości ustawiający objętość pęcherza, za nim pompa balastu z ograniczeniem wydajności.

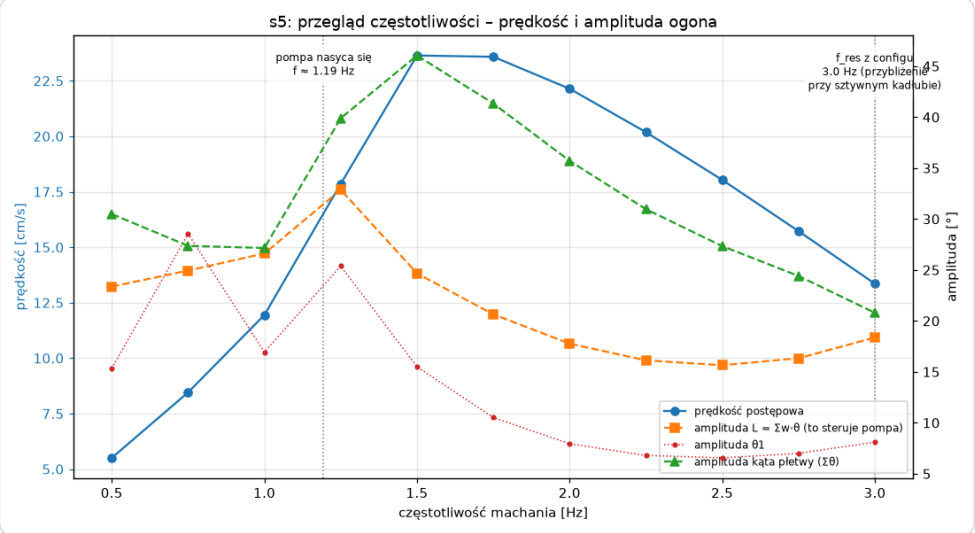
## Dane wejściowe

| KATEGORIA | UŻYWANE W DEMIE (`FISHSIM/CONFIG.PY`) | MOŻLIWE TEŻ W MUJOCO |
|---|---|---|
| **Środowisko** | woda ρ 1000 kg/m³, μ 0,0009 Pa·s, krok 2 ms (`implicitfast`), dno na −3 m | mapy wysokości, siatki terenu |
| **Geometria** | kadłub-elipsoida 0,25 × 0,10 × 0,10 m; 5 segmentów ogona × 40 mm zwężanych do 0,4; płetwa 70 × 6 × 120 mm | pliki MJCF lub URDF, siatki STL/OBJ |

| KATEGORIA | UŻYWANE W DEMIE (`FISHSIM/CONFIG.PY`) | MOŻLIWE TEŻ W MUJOCO |
|---|---|---|
| **Przeguby** | ±35°, sztywność napędzanych 0,3 N·m/rad, sztywność pasywnych z rezonansu 3 Hz, tłumienie 0,005 N·m·s/rad | dowolne typy przegubów, cięgna, więzy równościowe |
| **Model wody** | `fluidcoef` = blunt 0,5, slender 0,25, angular 1,5, Kutta 1,0, Magnus 1,0; tryb masy dodanej | odkształcalne bryły `flex` (zaobserwowane: model płynu ich nie uwzględniał; dokumentacja tego nie mówi) |
| **Hydraulika** | A_eff $4{\cdot}10^{-4}$ m², r_eff 0,05 m, C_h $8{\cdot}10^{-11}$ m³/Pa, p_max 50 kPa, komora 30 ml, Q_max 60 ml/s, τ_pump 30 ms | dowolny model w Pythonie sprzężony przez `ctrl` |
| **Balast i sterowanie** | pęcherz 0–40 ml przy 10 ml/s; wzmocnienia PID głębokości; ogon 2 Hz, A_V 8 ml, V_bias, K_v 10 $s^{-1}$ | czujniki (IMU, siła, dotyk), kamery, wsady GPU (MJX) do uczenia ze wzmocnieniem |
| **Wejście na żywo** | klawisze podglądu: ←/→ V_bias ±0,5 ml, ↑/↓ częstotliwość ±0,25 Hz, PgUp/PgDn głębokość ±0,25 m, spacja start/stop | dowolny callback w Pythonie |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **s1 pływanie prosto** | CSV + PNG: prędkość, pięć kątów przegubów, p_L, p_R, Δp, siła cięgna, sterowanie pompy | ≈22 cm/s przy 2 Hz po ≈2 s |
| **s2 woda wł./wył.** | prędkość z wodą, bez wody, z domyślną masą dodaną MuJoCo | 22 / 0 / 4,5 cm/s |
| **s3 skręt** | trajektorie XY dla V_bias od −3 do +3 ml | ±3 ml: 6,3°/s, promień ≈1,6 m |
| **s4 skoki głębokości** | głębokość, wartość zadana, objętość pęcherza, pochylenie | 1 m w ≈35 s płynąc, ≈6 s w zawisie; przeregulowanie ≈3 cm |
| **s5 przegląd częstotliwości** | prędkość, amplitudy, nasycenie pompy dla 0,5–3 Hz | maksimum 23,6 cm/s przy 1,5 Hz |
| **s6 prostowanie** | przechył, pochylenie i głębokość po przechyle 30° | pion w ≈0,3 s |
| **Raport modelu** | tekst: masy, objętości, wypór netto, CG/CB, zapasy stabilności kroku czasowego | dryf w zawisie 0,000 cm w 5 s |




Etap 6, przegląd częstotliwości. Prędkość osiąga maksimum przy 1,5 Hz, tuż powyżej 1,19 Hz, gdzie pompa zaczyna się nasycać.

## Etapy dema

`0` `check_api.py`: sprawdza API; kula o gęstości wody tonie, co pokazuje, że MuJoCo nie ma wbudowanego wyporu

`1` `show_model.py`: model statyczny i raport mas oraz wyporu

`2–5` `run_scenarios.py`: zawis i prostowanie, hydraulika offline, pływanie z diagnozą masy dodanej, skręt, skoki głębokości

`6` `sweep_frequency.py`: przegląd częstotliwości

`7` `run_viewer.py`: interaktywne sterowanie z klawiatury

## Ograniczenia

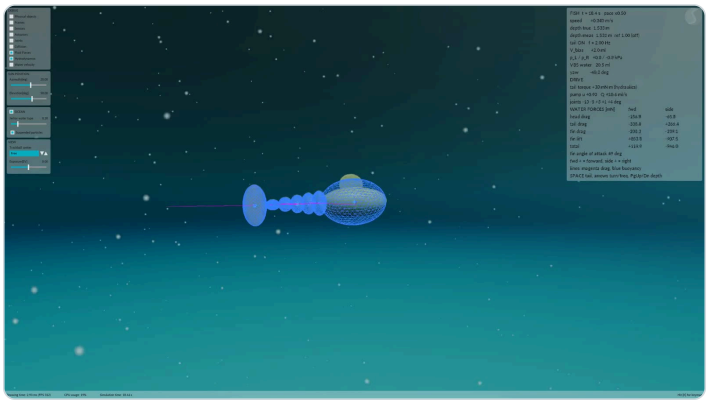
- Brak symulacji przepływu: brak śladu, wirów i oddziaływania części ciała przez wodę.
- Diagonalna masa dodana, liczona raz dla prostego ogona, więc ryba przyspiesza za szybko.
- Ogon to sztywne segmenty, a nie kontinuum. Hydraulika jest liniowa (stałe C_h, idealny zawór, bez strat).
- Brak fal i swobodnej powierzchni; przy powierzchni wypór jest wygaszany w paśmie ±5 cm jako łata.

# Stonefish

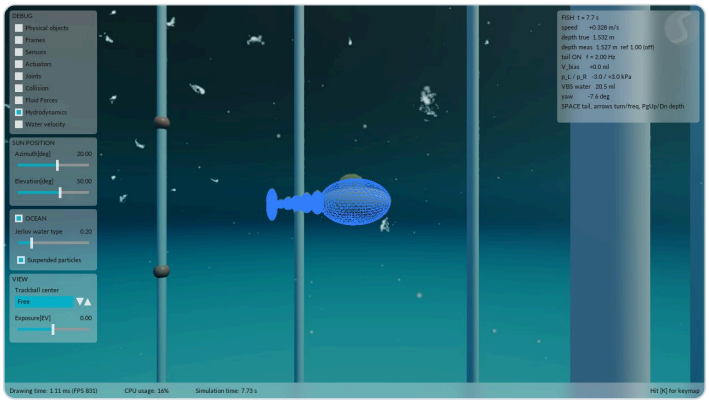
Symulator robotyki morskiej, który liczy siły wody z rzeczywistej siatki powierzchni i ma wbudowane czujniki, zbiorniki balastowe i prądy morskie. Ryba ma ten sam układ co w MuJoCo; dodatkowa fizyka jest napisana w C++.

| METODA | INTERFEJS | TESTOWANA WERSJA |
|---|---|---|
| Wielobryłowy Bullet (Featherstone) + hydrodynamika na trójkątach siatki | Aplikacja C++20 + scenariusze XML + konfiguracja JSON | Stonefish 1.5.0, budowany ze źródeł |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| Budowa CMake; OpenGL ≥ 4.3, SDL2, Freetype, GLM, OpenMP | 5–8× szybciej niż czas rzeczywisty w trybie konsoli przy 2000 Hz; GUI ≈800 FPS | GPL-3.0 |

## W programie: GUI i film demonstracyjny



**FILM** GUI Stonefish nagrane na żywo w 2560×1440: scenariusz s3 (skręt z V_bias = +2 ml) spowolniony do 0,5×, więc 25 s symulacji trwa 50 s. Panel po prawej dodaje napęd (moment hydrauliki, pompa, kąty przegubów) oraz siły wody w układzie głowy: opór głowy, ogona i płetwy, siłę nośną płetwy z modelu FinLift dema, sumę i kąt natarcia płetwy. Włączone są linie „Fluid Forces” biblioteki (magenta = opór, niebieski = wypór). Od 30 s do 40 s filmu włączony jest też widok „Hydrodynamics”. Plik filmu: `docs/media/stonefish_demo.mp4` (odtwarzany w wersji HTML).



**EKRAN** Widok debugowania „Hydrodynamics”: kształty, z których Stonefish liczy wypór, opór i masę dodaną, narysowane na niebiesko. Panel stanu (prawy górny róg) jest częścią dema.

## Zasada działania

- **Ciało.** Głowa i pięć ogniw ogona na przegubach obrotowych oraz płetwa. Siatki generuje `make_meshes.py` (320 trójkątów do fizyki, 5120 do wyświetlania), który też bilansuje masę i dobiera stalowy balast.
- **Woda z geometrii.** Wypór liczony jest z ciśnienia hydrostatycznego na każdym trójkącie. Opór ciśnieniowy działa na każdą ściankę, tarcie wzdłuż powierzchni, a masa dodana pochodzi z automatycznie dopasowanej kuli, walca lub elipsoidy.
- **Poprawki w C++.** Opór ścianek nie daje siły nośnej, więc `FinLift.cpp` dodaje siłę nośną płaskiej płyty na płetwie. Bez niej ryba osiąga tylko 2,7 cm/s zamiast 34 cm/s. Tarcie z biblioteki było ≈250× za duże i zastąpiono je prawem Blasiusa.

- **Napęd.** `Hydraulics.cpp` zawiera ten sam skupiony model pompy i komór co MuJoCo. `TailDriver.cpp` przykłada momenty τ_i = w_i·F − k_i·θ_i jako napęd wewnętrzny (akcja równa reakcji).
- **Czujniki i sterowanie.** Czujnik ciśnienia (50 Hz, σ 50 Pa) i IMU (100 Hz) są zaszumione. PID głębokości czyta filtrowane ciśnienie i ustawia objętość wody w VBS; regulator PI kursu na odchyleniu z IMU ustawia V_bias.
- **Ocean.** Dno, słupki jako punkty odniesienia, typ wody i jednorodny prąd.

## Dane wejściowe

| KATEGORIA | UŻYWANE W DEMIE | MOŻLIWE TEŻ W STONEFISH |
|---|---|---|
| **Plik konfiguracji** | `config/default.json`, łatki scenariuszy `sN_*.json`, pojedyncze nadpisania `--set klucz=wartość` (bez rekompilacji) | dowolny parametr udostępniony przez aplikację |
| **Scenariusz XML** | `fish_base.scn`: ogniwa, przeguby, silniki, VBS, czujnik ciśnienia, IMU, enkodery; `world_base.scn`: dno, ocean, prąd | fale geometryczne (liczone na GPU; oddziaływanie z bryłami wciąż eksperymentalne), inne typy prądów, własne współczynniki hydrodynamiczne |
| **Geometria i materiały** | siatki OBJ; gęstość kadłuba 600, silikon 1100, stalowy balast 7850 kg/m³ (dobierany automatycznie) | dowolna geometria OBJ |
| **Hydraulika i rytm** | te same wartości co w MuJoCo: Q_max 60 ml/s, A_V 8 ml, f 2 Hz, p_max 50 kPa; nachylenie siły nośnej płetwy 2,8/rad | pędniki, stery, serwa |
| **Czujniki** | ciśnienie 50 Hz z σ 50 Pa; IMU 100 Hz z szumem kąta, żyroskopu i akcelerometru | kamery, sonar, DVL, GPS, kamera termowizyjna |
| **Sterowanie i wejście na żywo** | harmonogram głębokości zadanej, PI kursu; klawisze GUI dla CPG, V_bias, częstotliwości i głębokości | interfejs ROS 2 (`stonefish_ros2`), tu nieużyty |
| **Symulacja** | 2000 kroków/s (2 ms było niestabilne), hydrodynamika w każdym kroku, log co 20 kroków | tryb czasu rzeczywistego lub wsadowy |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **Log CSV, 100 Hz** | ≈70 kolumn: poza, prędkości, odczyty czujników, θ/ω/τ przegubów, pompa, komory, VBS, siły na częściach ciała | pełne logi w `results/logs/` |
| **s1 zawis / prostowanie** | dryf głębokości; przechył po wychyleniu o 30° | 0,21 mm w 10 s; 30° → 10° w 5 s |
| **s2 pływanie** | prędkość z siłą nośną płetwy i bez niej, ogon zablokowany; bilans sił | 34 cm/s; 2,7 cm/s bez siły nośnej; siła nośna płetwy 0,51 N |
| **s3 skręt** | promień skrętu dla V_bias ±1/±2/±3 ml | 20,4 / 10,1 / 6,6 m |
| **s4 głębokość** | głębokość prawdziwa vs mierzona zaszumionym czujnikiem | 1 → 3 m: przeregulowanie 3,3 cm, w paśmie ±5 cm po 21 s |
| **s5 prąd** | dryf w bocznym prądzie 5 cm/s, z regulacją kursu i bez | kurs utrzymany na +2,9°, a mimo to 1 m dryfu w bok w 30 s |
| **s6 przegląd, GUI** | prędkość vs częstotliwość; `summary.json`; okno 3D | maksimum 33,9 cm/s przy 2 Hz |




GUI Stonefish w scenariuszu s2: panel po prawej pokazuje na żywo prędkość, głębokość prawdziwą i mierzoną, ciśnienia w komorach i objętość VBS.

## Etapy dema

1–2 Budowa Stonefish; generowanie siatek i bilansu masy (`tools/make_meshes.py`)

3 s1: zawis i prostowanie

4 s2: przeguby i hydraulika, diagnoza źródła ciągu

5 s3–s5: skręt, głębokość na zaszumionym czujniku, boczny prąd

6 s6: przegląd częstotliwości; `t_internal` sprawdza, że napęd jest wewnętrzny

7 ROS 2: pominięty

## Ograniczenia

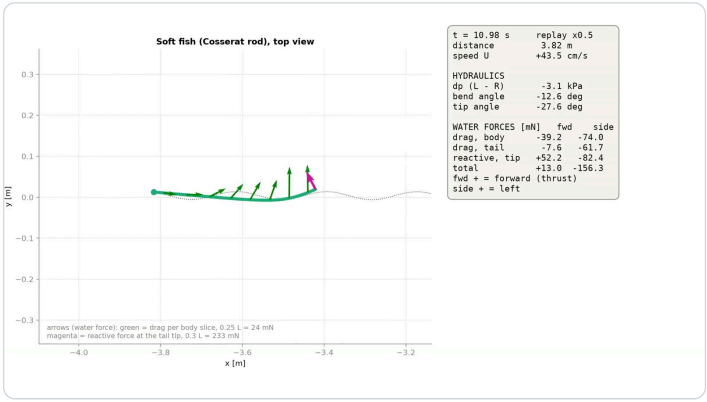
- Ogon ze sztywnych segmentów; quasi-statyczna hydrodynamika bez śladu wirowego, bez opóźnionego przeciągnięcia i bez oddziaływania między bryłami.
- Masa dodana uśredniona po osiach, bez dodanej bezwładności wokół osi przechyłu; tarcie zlinearyzowane przy 0,2 m/s.
- GUI wymaga GPU. Udokumentowano osiem pułapek biblioteki w wersji 1.5 wraz z obejściami.

# PyElastica

Cała ryba to jeden miękki pręt Cosseratów: linia środkowa z obracającym się układem odniesienia, która może się zginać, skręcać, ścinać i rozciągać. To tutaj najtańszy sposób na symulację ciągle odkształcającego się ciała, które pływa swobodnie.

| METODA | INTERFEJS | TESTOWANA WERSJA |
|---|---|---|
| Teoria prętów Cosseratów, jawny symplektyczny Position Verlet | Czysty Python, `import elastica as ea`, jądra numba | PyElastica 1.0.0, numba 0.68, Python 3.14 (działa; PyElastica oficjalnie wymienia 3.10–3.13) |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| `pip install -r requirements.txt`, kilka minut | ≈27 µs na krok pręta ze 100 elementów; 9–70 s na etap | MIT |

## W programie: GUI i film demonstracyjny




**FILM** Swobodne pływanie przy 2 Hz, widok z góry: 15 s symulacji w tempie 0,5× (30 s), wyrenderowane w 2560×1440 przez `scripts/animate.py` w gałęzi pyelastica. PyElastica nie ma GUI, więc matplotlib rysuje zapisane położenia pręta i odczyty zapisane w etapie 4. Są to prędkość, Δp, kąt zgięcia i kąt końcówki oraz siły wody w układzie ryby: opór całego ciała i tylnej połowy (ogona), siła reaktywna Lighthilla na końcówce i suma. Zielone strzałki to opór na każdym odcinku ciała, strzałka magenta to siła reaktywna na końcówce, we własnej skali. Plik filmu: `docs/media/pyelastica_free_swim.mp4` (odtwarzany w wersji HTML).

## Zasada działania

- **Ciało.** Pręt 0,40 m z 50 elementami. Każdy element ma przekrój eliptyczny (z tabeli 7 stacji), więc ogon jest miękki na boki i sztywny w pionie. Głowa jest 20× sztywniejsza.
- **Napęd od środka.** Ciśnienie w komorze zmienia krzywiznę spoczynkową strefy komór (0,20–0,32 m): κ_rest = −A_r·Δp / B. Napęd nie daje siły wypadkowej; w próżni środek masy dryfuje tylko o $10^{-15}$ m.
- **Hydraulika.** Ten sam skupiony model pompy, komór i zaworu co w MuJoCo i SOFA, liczony co 0,2 ms i sprzężony przez kąt zgięcia strefy komór.
- **Woda.** Własna siła: kwadratowy opór Morisona na jednostkę długości plus siła reakcyjna Lighthilla (duże amplitudy) na krawędzi spływu. Wbudowany model Stokesa dla ciał smukłych zostawiono tylko po to, by pokazać, że przy tej liczbie Reynoldsa (≈$1{,}8\cdot10^{5}$) jest błędny: jego siła jest ≈400× za mała.
- **Wyporność i głębokość.** Ciężar i wypór dla każdego elementu, moment prostujący od środka ciężkości poniżej osi, pęcherz i kaskadowy PID głębokości przeniesiony z MuJoCo.
- **Krok czasowy.** ≈19 µs, z jawnego warunku stabilności pręta.

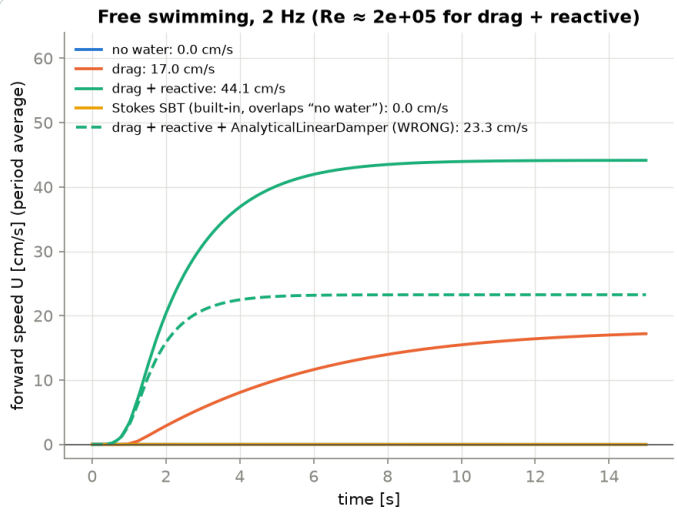
## Dane wejściowe

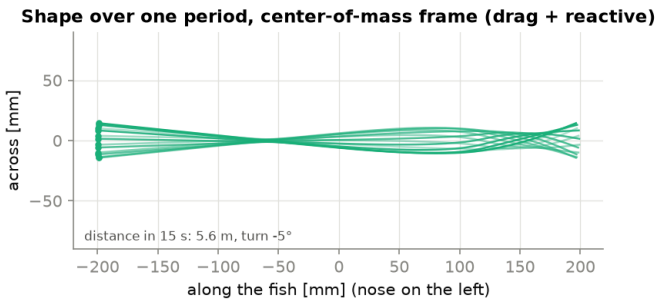
| KATEGORIA | UŻYWANE W DEMIE (`FISHROD/CONFIG.PY`) | MOŻLIWE TEŻ W PYELASTICA |
|---|---|---|
| **Pręt** | L 0,40 m, ρ 1050 kg/m³, E 0,5 MPa, ν 0,5, 50 elementów | kilka prętów połączonych przegubami, bryły sztywne |
| **Kształt** | półszerokość i półwysokość w 7 stacjach (24 × 34 mm w środku ciała), elipsa lub koło, sztywna głowa | dowolny promień i sztywność dla każdego elementu |
| **Hydraulika** | A_eff $4{,}3\cdot10^{-4}$ m², r_eff 5,5 mm, C_h $4{,}5\cdot10^{-10}$ m³/Pa, p_max 50 kPa, Q_max 160 ml/s, f 2 Hz, A_V 8 ml | wbudowane momenty mięśniowe, momenty jednorodne |
| **Woda** | model: none, drag, drag+reactive, stokes_sbt; C_n 1,2, C_t 0,01 | sprzężenie z solverem przepływu metodą brzegu zanurzonego (SophT) |

| KATEGORIA | UŻYWANE W DEMIE (`FISHROD/CONFIG.PY`) | MOŻLIWE TEŻ W PYELASTICA |
|---|---|---|
| **Wyporność i sterowanie** | CG 5 mm poniżej osi, pęcherz 0–40 ml przy 10 ml/s, PID przy 100 Hz, harmonogram głębokości | kontakt i tarcie z dnem |
| **Ustawienia etapów** | siatki przeglądu (f 0,5–3 Hz × E ×0,5/1/2), czasy trwania, typ tłumienia | inne warunki brzegowe i tłumiki |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **e1 walidacja** | wspornik vs teoria, częstotliwość własna, dryf energii dla n = 25/50/100 | n = 50: błąd końca 1,05 %, błąd $\omega_1$ 1,8 % |
| **e2 napęd** | zgięcie vs ciśnienie i vs objętość; test napędu wewnętrznego | 8 ml → 15,8 kPa, 0,377 rad |
| **e3 na uwięzi** | amplituda końca, faza, ciąg i siła boczna dla każdego modelu wody | ciąg 149 mN z siłą reakcyjną, 5,9 mN z samym oporem |
| **e4 swobodne pływanie** | prędkość w czasie; kształt ciała w jednym okresie; `free_swim.gif` | 44,1 cm/s (1,1 długości ciała/s); 17 cm/s z samym oporem |
| **e5 przegląd f × E** | prędkość, liczba Strouhala, częstotliwość własna ogona | największy ogon przy 3 Hz: 55 cm/s; f_n 1,89/2,60/3,64 Hz |
| **e6 głębokość** | głębokość, objętość pęcherza, pochylenie, przechył, prędkość | 1,0 → 1,5 m ustala się w 9,9 s, przeregulowanie 23 cm |




Etap 4: wybór modelu wody decyduje o wyniku. Bez siły reakcyjnej (Lighthilla) ryba osiąga 17 cm/s; model Stokesa daje zero.

## Etapy dema

1. Walidacja pręta względem teorii belek, zbieżność w 1/n
2. Napęd przez krzywiznę spoczynkową i sprzężenie hydrauliczne; test napędu wewnętrznego w próżni
3. Ogon na uwięzi z trzema modelami wody; ciąg z bilansu pędu
4. Swobodne pływanie w płaszczyźnie, porównanie modeli wody i tłumienia
5. Przegląd częstotliwość × sztywność, rezonans, liczba Strouhala
6. 3D z grawitacją, wyporem, pęcherzem i PID głębokości w trakcie pływania
7. Eksport sztywności do MuJoCo: niewykonany

## Ograniczenia

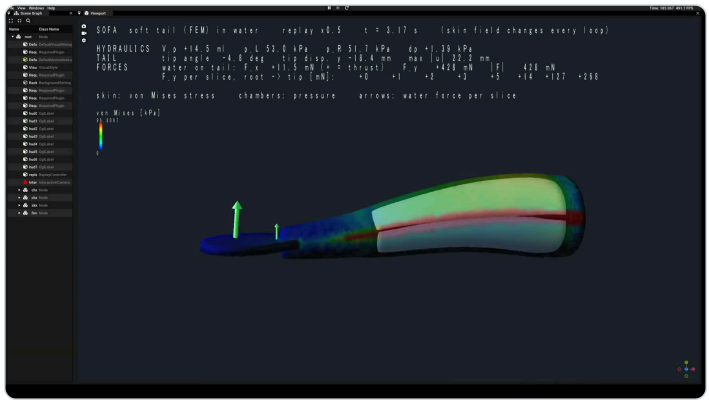
- Lokalny model wody bez śladu: liczba Strouhala jest za niska (0,12–0,2), a prędkość zawyżona.
- Brak rozłożonej masy dodanej, więc częstotliwość własna w wodzie jest za wysoka. Przekrój to elipsa, a nie cienka płyta płetwy.
- Komora sprowadzona do A_r·Δp i C_h; sprężystość liniowa. Bardzo sztywna sprężyna hydrauliczna psuje jawne sprzężenie.

# SOFA + SoftRobots

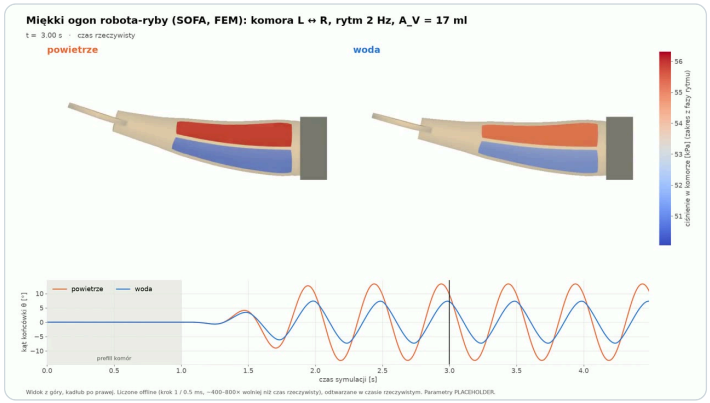
Trójwymiarowy model elementów skończonych silikonowego ogona z prawdziwymi wnękami komór. Pompa zadaje objętość komory, a SOFA wylicza potrzebne ciśnienie. Ogon jest utwierdzony u nasady, jak na stanowisku do pomiaru ciągu.

| METODA | INTERFEJS | TESTOWANA WERSJA |
|---|---|---|
| Korotacyjna MES na czworościanach + więzy Lagrange'a dla komór | Sceny SofaPython3 (`createScene`), silnik w C++ | Binaria SOFA v26.06.00 (SoftRobots w zestawie) |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| `scripts/setup.sh`: ≈2 min + 230 MB, buduje wtyczkę CHOLMOD | 100–800× wolniej niż czas rzeczywisty (siatka zgrubna, jeden rdzeń) | LGPL-2.1+ (SOFA), LGPL-3.0 (SoftRobots) |

## W programie: GUI i film demonstracyjny



**FILM** GUI SOFA (ImGui) w 2560×1440 odtwarza przebieg w wodzie w tempie 0,5×; sam przebieg policzono offline (`scripts/record.py`, ≈30 min). Tekst pokazuje hydraulikę (V_p, p_L, p_R, Δp), ogon (kąt końcówki, przemieszczenie końcówki, max |u|) oraz siłę wody na ogon, łącznie i na odcinki. Zielone strzałki rysują siłę na każdym odcinku, a komory są pokolorowane ciśnieniem. Kolor skóry zmienia się w każdej pętli, każdy z własną legendą: przemieszczenie [mm], obciążenie wodą [Pa], potem naprężenie von Misesa [kPa]. Naprężenie jest liczone z zapisanych położeń węzłów tym samym modelem korotacyjnym co MES. Plik filmu: `docs/media/sofa_gui_replay.mp4` (odtwarzany w wersji HTML).




**FILM** Film renderowany z gałęzi sofa (PyVista, 6,5 s): powietrze i woda obok siebie, pod spodem kąt końcówki, najpierw w czasie rzeczywistym, potem ostatni cykl 4× wolniej. Plik filmu: `docs/media/sofa_flapping.mp4` (odtwarzany w wersji HTML).



**EKRAN** Interfejs ImGui SOFA: graf sceny (komponenty sceny ogona) po lewej, okno widoku po prawej.

## Zasada działania

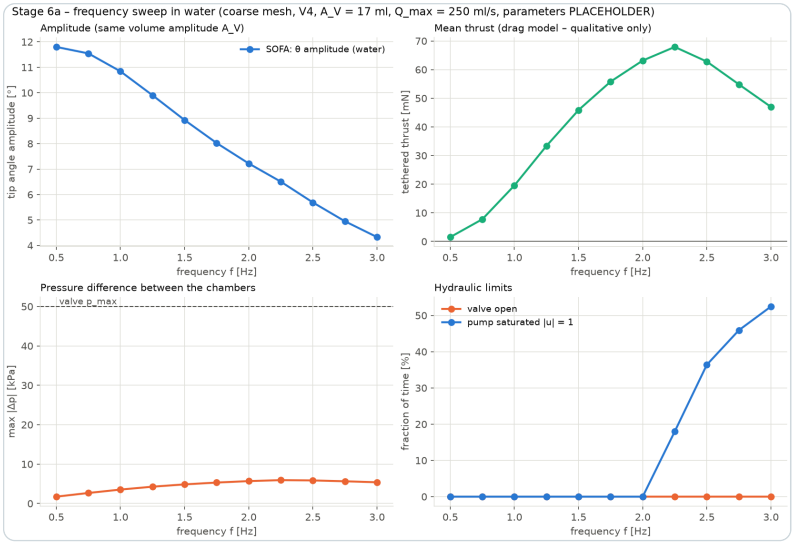
- **Siatka.** gmsh buduje ogon z dwiema wnękami. Poziom zgrubny ma 28 112 czworościanów i 7 744 węzły; poziom dokładny 142 096 czworościanów.
- **Materiał.** Korotacyjna sprężystość liniowa (duże obroty, małe odkształcenia), E 0,3 MPa, v 0,45. Domyślny wariant V4 dodaje w przegrodzie kręgosłup 20× sztywniejszy i co 4 mm obwodowe włókna pracujące tylko na rozciąganie.
- **Komory.** `SurfacePressureConstraint` w trybie przyrostu objętości: pompa ustawia objętość, a solver więzów zwraca ciśnienie.
- **Solver.** Niejawny Euler, solver więzów Gaussa–Seidela i rzadki rozkład Cholesky'ego CHOLMOD (4–15× szybszy od LDL). Krzywe statyczne wyznacza się pseudo-statycznie, trzymając każdy punkt, aż ruch wygaśnie.
- **Woda.** Własny kwadratowy opór normalny i tarcie styczne na każdym trójkącie skóry, przykładane jawnie; ciężar silikonu pomniejszony o wypór. Brak masy dodanej, siły nośnej i śladu.
- **Hydraulika.** Port rytmu z MuJoCo z kompensacją opóźnienia pompy (bez niej objętość przestrzeliwuje o 16 % przy 2 Hz), obieg zamknięty i wstępne napełnienie 20 ml.

## Dane wejściowe

| KATEGORIA | UŻYWANE W DEMIE (`FISHSOFA/CONFIG.PY`) | MOŻLIWE TEŻ W SOFA |
|---|---|---|
| **Geometria** | ogon 0,20 m, przekrój 60 × 80 mm zwężany do 0,4, płetwa 70 × 120 × 6 mm; dwie komory po 0,12 m, ścianka 4 mm, przegroda 4 mm (po 91 ml) | dowolna siatka Gmsh, STL, OBJ lub VTK; sceny XML `.scn` |
| **Materiał** | E $3 \cdot 10^5$ Pa, v 0,45, ρ 1100; kręgosłup E × 20; włókna $2 \cdot 10^5$ N/m co 4 mm | materiały hipersprężyste, belki (BeamAdapter) |
| **Napęd** | f 2 Hz, A_V 17 ml, napełnienie wstępne 20 ml, Q_max 250 ml/s, p_max 50 kPa | sterowanie ciśnieniem, aktuatory cięgnowe, sterowanie odwrotne (SoftRobots.Inverse) |
| **Środowisko** | powietrze lub woda; współczynniki oporu C_n 1,0, C_t 0,01 | kontakt i kolizje |
| **Numeryka** | krok 2 ms (0,5–1 ms w wodzie), tłumienie Rayleigha, solver cholmod/ldl/cg/warp, poziom siatki test/coarse/medium/fine | redukcja rzędu modelu (>100× szybciej wg Goury & Duriez 2018; wymaga treningu offline) |
| **Sterowanie uruchomieniem** | zmienne środowiskowe `FISHSOFA_LEVEL`, `_MODE`, `_ENV`, `_WORKERS` | interakcja w GUI |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **s1 siatka i ugięcie** | raport jakości siatki; ugięcie końca pod własnym ciężarem na trzech siatkach | −40,5 mm (zgrubna) do −41,2 mm (dokładna) |
| **s2a warianty konstrukcji** | zgięcie dla: jednorodnego, z kręgosłupem, grubej ścianki, włókien, kręgosłupa + włókien | 1,3° jednorodny vs 31° dla V4 |
| **s2 krzywa p–V** | ciśnienie i kąt końca vs objętość komory, zbieżność siatki | 50 ml: 16,1 kPa i 15,4° (siatka dokładna) |
| **s3 symetria** | komora lewa vs prawa | błąd ≤ 0,75 % w kącie |
| **s4 machanie w powietrzu** | kąt, ciśnienia, zachowanie objętości, iteracje solvera | ±12,7° przy 2 Hz, $\lvert\Delta p\rvert \le 5{,}7$ kPa |
| **s5 woda vs powietrze** | amplituda, faza, ciąg na uwięzi, siła boczna, moc oporu | ±7,3° w wodzie (0,55× powietrza); ciąg 66,7 mN |
| **s6 przeglądy, wideo** | przeglądy częstotliwości i modułu Younga; `flapping.mp4` 1080p60 | maks. ciąg 67,9 mN przy 2,25 Hz |




Etap 6 w wodzie. Amplituda maleje równomiernie z częstotliwością (brak rezonansu w wodzie), ciąg ma maksimum przy 2,25 Hz, a powyżej 2 Hz pompa się nasyca. Ciąg pochodzi z modelu oporu i jest jakościowy.

## Etapy dema

| | |
|---|---|
| 0 | Sprawdzenie komponentów; znaczenie, znak i jednostki więzu objętości komory (≈2,5 min) |
| 1 | Siatki i ugięcie pod własnym ciężarem (≈10 min) |
| 2 | Warianty V0–V4; krzywe p–V i kąta, zbieżność siatki (≈35 min) |
| 3 | Symetria lewo/prawo (≈3 min) |
| 4 | Antagonistyczne machanie w powietrzu (≈25 min) |
| 5 | Woda vs powietrze, ciąg na uwięzi (30–60 min) |
| 6 | Przeglądy częstotliwości i sztywności, 19 symulacji (≈1 h na 12 wątkach) |

## Ograniczenia

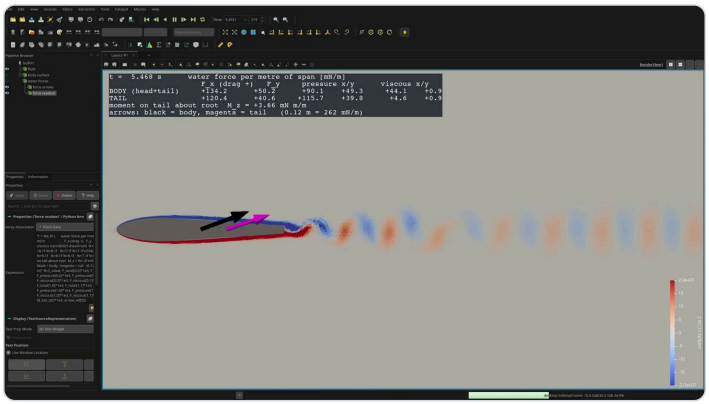
- Brak przepływu i siły reakcyjnej, więc ciąg jest jakościowy. Ogon jest utwierdzony; ryba nie pływa.
- Liniowe czworościany blokują się w cienkich ściankach, co zawyża ciśnienie o 15–25 %. Materiał jest korotacyjny liniowy, nie hipersprężysty.
- Wolno: jawny opór ogranicza krok do 0,5 ms w wodzie. W naszych testach SOFA master (v26.12-dev, niewydany) dawał drgającą lub rozbiegającą się komorę ze sztywnymi włóknami, więc demo zostaje na v26.06 (szczegóły w `SOFA/README.md`; zgłoszenia upstream brak).
- Eksport równoważnych sztywności przegubów do MuJoCo (etap 7) nie został wykonany.

# OpenFOAM + preCICE + CalculiX

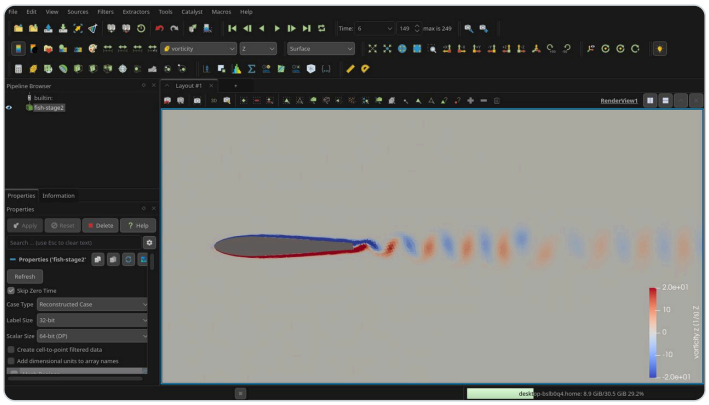
Jedyne demo, które rozwiązuje rzeczywisty przepływ wody. Solver CFD i solver MES konstrukcji wymieniają siły i przemieszczenia w każdym kroku czasu, aż się zgodzą. To przekrój 2D ogona z nieruchomą głową.

| METODA | INTERFEJS | TESTOWANE WERSJE |
|---|---|---|
| Podzielona FSI: objętości skończone + nieliniowa MES, sprzężenie niejawne | Słowniki OpenFOAM, `.inp` CalculiX, XML preCICE; narzędzia w Bash i Pythonie | OpenFOAM v2606, CalculiX 2.20 (istnieje 2.23; adapter preCICE v2.20.2 obsługuje 2.20), preCICE 3.4.1 |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| `setup.sh`, ≈1,5 h, głównie budowa ze źródeł | 0,6–1 s na okno 2,5 ms; etapy 4–6 trwają 3 h 47 min | GPL-3.0 (OpenFOAM), GPL-2.0+ (CalculiX), LGPL-3.0 (preCICE) |

## W programie: GUI i film demonstracyjny



**FILM** ParaView 6.1 w 2560×1440 odtwarza etap 2 (ciało sztywne, U = 0,2 m/s, 10 s przepływu) w tempie ok. 0,5×. `tools/pv_gui.py` dodaje ramkę tekstu, która podąża za czasem animacji i jest czytana z obiektów funkcyjnych `forces`. Pokazuje siłę wody na ciało i na ogon (suma, część ciśnieniowa i lepka, w mN na metr rozpiętości) oraz moment na ogonie względem nasady. Strzałki pokazują siłę całkowitą w środkach ciała (czarna) i ogona (magenta). Przypadek policzono ponownie na potrzeby raportu w OpenFOAM v2512 w oficjalnym kontenerze: 20 665 komórek, 90 s na 4 rdzeniach. Plik filmu: `docs/media/openfoam_paraview_demo.mp4` (odtwarzany w wersji HTML).



**EKRAN** ParaView z wczytanym przypadkiem OpenFOAM: przeglądarka potoku i właściwości czytnika po lewej, pole wirowości w chwili t = 6 s po prawej.

## Zasada działania

- **Płyn.** `pimpleFoam` rozwiązuje nieściśliwe laminarne równania Naviera–Stokesa w 2D (jedna komórka w głąb, więc siły są na metr rozpiętości) na siatce odkształcającej się razem z ogonem.
- **Ciało stałe.** CalculiX liczy dynamiczną nieliniową MES (duże odkształcenia, liniowo sprężysty silikon, E 0,3 MPa) ogona z komorami w stylu PneuNet.
- **Sprzężenie.** preCICE odwzorowuje siły płynu na ciało stałe i przemieszczenia ciała na siatkę płynu, a każde okno 2,5 ms powtarza metodą quasi-Newtona IQN-ILS, aż oba się zgodzą (średnio 4,7 iteracji).

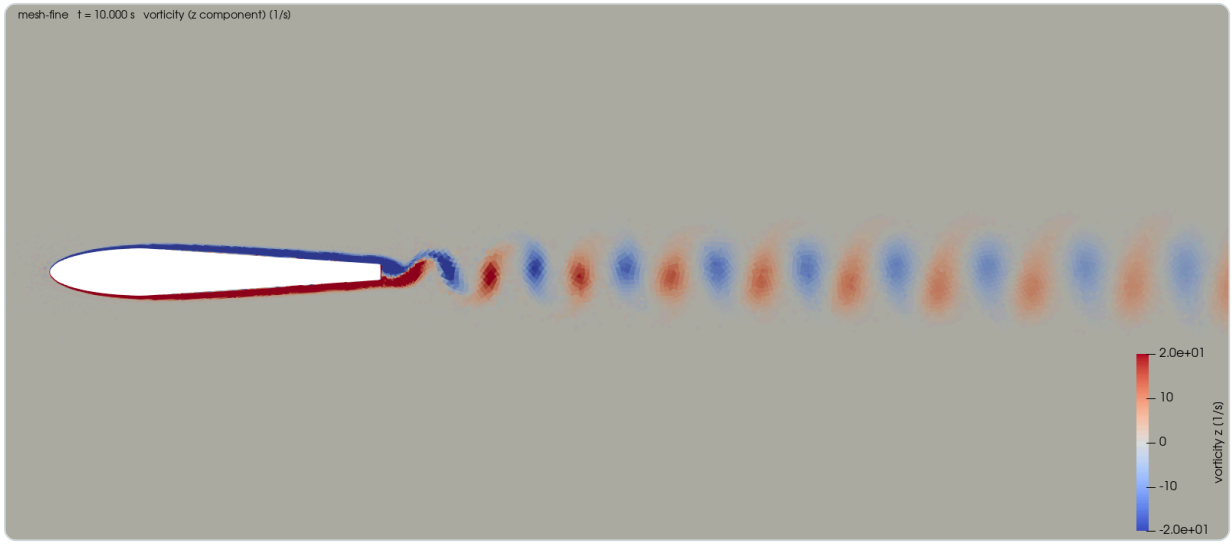
- **Dlaczego niejawnie.** Silikon ma gęstość prawie jak woda, więc sprzężenie jawne jest niestabilne dla każdego kroku czasu (efekt masy dodanej). Etap 3 pokazuje wzrost siły 13× na okno.
- **Napęd.** Zadane ciśnienie w komorze $p_L = P0 \cdot \max(0, \sin 2\pi f t)$, z przeciwną połówką fali w $p_R$. Wejściem jest ciśnienie, a nie objętość.
- **Ciąg.** Uśredniona po cyklu siła osiowa na nieruchomym ciele (głowa + ogon).

## Dane wejściowe

| KATEGORIA | UŻYWANE W DEMIE | MOŻLIWE TEŻ W TYM ZESTAWIE |
|---|---|---|
| **Geometria (`params.py`)** | ogon 0,15 m, grubość 30 → 10 mm, komory na 10–110 mm, ścianki 3 mm, 10 cel na stronę; głowa pół-elipsa 60 mm; domena 3L przed, 8L za | pełna geometria 3D |
| **Materiały i woda** | E $3 \cdot 10^5$ Pa, ν 0,45, ρ 1070 kg/m³; woda ρ 1000, ν $10^{-6}$ m²/s | materiał hipersprężysty, tłumienie materiałowe (CalculiX) |
| **Przypadek (`new-fsi-case.sh`)** | prędkość napływu U, krok czasu, czas końcowy, P0, częstotliwość, sprzężenie niejawne/jawne | swobodne ciało z solverem 6 stopni swobody |
| **Ustawienia OpenFOAM** | PIMPLE + GAMG, laplasjanowy ruch siatki, obiekty funkcyjne sił i wirowości, rampa na wlocie, boki z poślizgiem | turbulencja RANS/LES, siatki overset lub przesiatkowanie dla dużych amplitud |
| **Ustawienia CalculiX** | `fsi.inp`: dynamika nieliniowa, HHT α −0,2, solver PARDISO, tablice amplitudy ciśnienia | kontakt ścianek komór |
| **Ustawienia preCICE** | odwzorowanie RBF (promień 5 mm), IQN-ILS, zbieżność $10^{-3}$, punkt obserwacji końca | kolejni uczestnicy, np. model hydrauliki sterowany objętością |
| **Siatka** | gmsh, zagęszczenie przez `MESH_SCALE` (0,7 / 1,0 / 1,4) | dowolny generator siatek OpenFOAM |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **Pola przepływu** | prędkość, ciśnienie, wirowość, średnie czasowe dla ParaView | ścieżka wirowa Kármána za ciałem |
| **Siły** | siła i moment ciśnieniowe i lepkie na każdym fragmencie brzegu (głowa, ogon) | ciało sztywne przy 0,2 m/s: opór 0,13 N/m, ⅔ ciśnieniowy |
| **E1 suchy ogon** | ugięcie końca vs ciśnienie, symetria L/R | 1,2 mm/kPa; 24 mm (16 % L) przy 20 kPa |
| **E3 sprzężenie** | stabilność jawnego vs niejawnego | jawne się rozbiega; niejawne stabilne |
| **E4 z napędem** | amplituda końca w wodzie vs na sucho, ciąg na cykl | 28,3 mm w wodzie vs 20 mm na sucho przy 1 Hz |
| **E5 napływ** | średnia siła osiowa i liczba Strouhala vs prędkość | opór wypadkowy przy 0,05–0,2 m/s; zero dopiero ok. 0,02 m/s |
| **E6 częstotliwość** | amplituda i ciąg vs częstotliwość | rezonans w wodzie ≈1 Hz (na sucho 3,39 Hz) |




Etap 2, siatka dokładna, U = 0,2 m/s: ślad wirowy, którego nie pokaże żadne z pozostałych pięciu narzędzi.

## Etapy dema

0 Oficjalny tutorial perpendicular-flap do sprawdzenia instalacji (34 s)

1 Samo ciało stałe: ugięcie vs ciśnienie (≈10 s)

2 Sam płyn, ciało sztywne: opór i zbieżność siatki (7 min)

3 Ogon pasywny: sprzężenie jawne vs niejawne (10 min)

4 Ogon z napędem w stojącej wodzie vs na sucho (16–80 min)

5 Napływ 0,05/0,1/0,2 m/s (≈30 min każdy)

6 Przegląd częstotliwości 0,5–2 Hz (16–102 min każdy)

## Ograniczenia

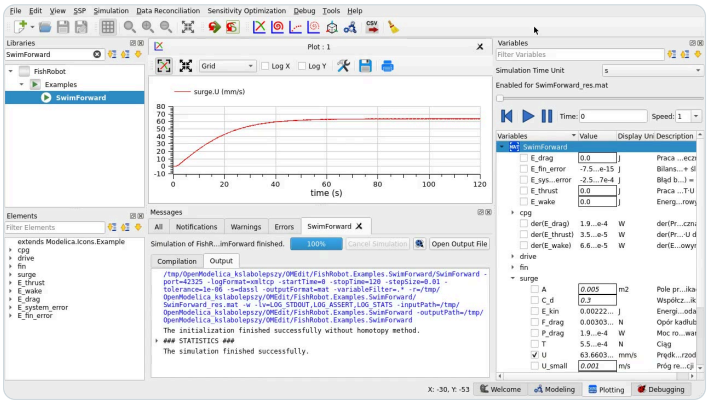
- 2D: brak wirów krańcowych, a siły są na metr rozpiętości. Przepływ laminarny przy $Re \approx 4 \cdot 10^4$.
- Nieruchoma głowa, więc brak swobodnego pływania i odrzutu. Napęd ciśnieniem, nie objętością. Brak tłumienia materiałowego.
- Odkształcanie siatki ogranicza amplitudę końca do ≈15 % L. Dwie jednorodne komory dają falę stojącą i tępy koniec, więc ogon generuje opór wypadkowy.
- Wolne i najtrudniejsze w instalacji z sześciu.

# OpenModelica

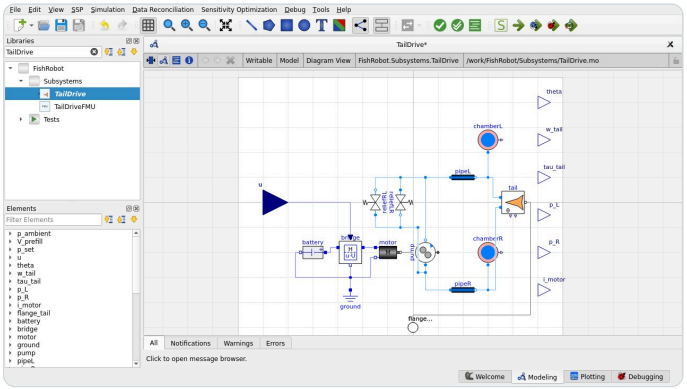
Równaniowy model całego układu napędowego, od baterii przez silnik, pompę, przewody i komory do ogona o jednym stopniu swobody. To jedyne demo z elektryką, bilansem energii, eksportem FMU i pełnym procesem kalibracji.

| METODA | INTERFEJS | TESTOWANA WERSJA |
|---|---|---|
| Akauzalne równania Modelica → DAE → kod C, solver o zmiennym kroku | Modelica (GUI OMEdit), Python przez OMPython i FMPy | OpenModelica 1.27.1, MSL 4.1.0 |
| **INSTALACJA** | **SZYBKOŚĆ** | **LICENCJA*** |
| repozytorium apt (`install_system.sh`) + venv (`install_user.sh`) | 16 symulacji ≈0,2 s po ≈1 s kompilacji | OSMC-PL 1.8 lub AGPL-3.0 (OpenModelica), BSD-3 (MSL) |

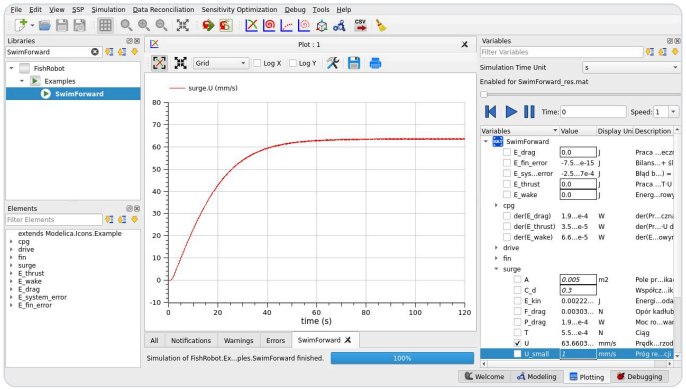
## W programie: GUI i film demonstracyjny



**FILM** OMEdit (OpenModelica 1.27.1, oficjalny kontener): przykład SwimForward jest symulowany (kompilacja i obliczenia ≈5 s; oczekiwanie wycięte), wykreślana jest prędkość U, a potem dodawany ciąg T. Plik filmu: `docs/media/openmodelica_demo.mp4` (odtwarzany w wersji HTML).



**EKRAN** Widok diagramu `FishRobot.Subsystems.TailDrive` w OMEdit: bateria, mostek H, silnik, pompa, przewody, dwie komory, zawory przelewowe i ogon.



**EKRAN** Widok wykresów OMEdit po symulacji SwimForward: prędkość ustala się na 63,7 mm/s, czyli 6,4 cm/s jak w README.

## Zasada działania

- **Komponenty akauzalne.** Części łączy się złączami fizycznymi. Własny port hydrauliczny niesie ciśnienie p i przepływ V_flow, których iloczyn to moc. Kompilator sam porządkuje równania, więc nie ma okablowania wejście/wyjście.
- **Łańcuch napędu.** Bateria 2S LiPo 7,4 V → mostek H → silnik DC → pompa zębata ($Q = D \cdot \omega - k_{leak} \cdot \Delta p$) → przewody (tarcie laminarne plus turbulentne Haalanda, straty miejscowe) → komory z nieliniową tablicą p–V → zawory przelewowe.
- **Ogon i płetwa.** Ogon to tłok obrotowy o jednym stopniu swobody: $(J + J_{added})\cdot\ddot{\theta} = \tau - k\theta - c\dot{\theta} - c_h|\theta|\theta$. Ciąg pochodzi ze wzoru Lighthilla dla sztywnej płetwy i napędza równanie ruchu postępowego.
- **Balast.** Silnik ze śrubą pociągową przesuwa tłok strzykawki; równanie ruchu pionowego i kaskadowy PID głębokości zamykają pętlę.
- **Energia.** Każda strata jest całkowana, a bilans domyka się z dokładnością ≈$10^{-6}$.

- **Eksport i kalibracja.** `TailDrive` eksportuje się jako FMU 2.0 do kosymulacji. Dziesięć modeli stanowisk i `calibrate.py` (najmniejsze kwadraty) identyfikują parametry krok po kroku, od silnika do wody.

## Dane wejściowe

| KATEGORIA | UŻYWANE W DEMIE | MOŻLIWE TEŻ W MODELICE |
|---|---|---|
| **Elektryka** | bateria 7,4 V, R_int 0,05 Ω, 16,3 Wh; silnik R 1 Ω, L 0,5 mH, k 0,01 N·m/A, bezwładność wirnika | napięcie zależne od stanu naładowania, straty PWM, modele cieplne |
| **Hydraulika** | pompa 0,3 ml/obr., przecieki, η 0,8; przewód 0,2 m × 4 mm, ζ 1,5, chropowatość 25 µm; zawór 50 kPa; tablica p–V komory (11 punktów lub plik CSV) | `Modelica.Fluid` i media, ściśliwe lub dwufazowe (wariant jest już w `HydraulicsFluid/`) |
| **Ogon i płetwa** | chłonność $2{\cdot}10^{-5}$ m³/rad, k 2 N·m/rad, bezwładność, bezwładność dodana, tłumienie; rozpiętość płetwy 0,06 m, C_T 1,0 | krzywa p–V z SOFA albo z pomiaru |
| **Ciało i balast** | m 1 kg, masa dodana, C_d 0,3, pole 0,005 m²; strzykawka 20 mm × 40 mm (12,5 ml), skok śruby 1 mm | biblioteka wielobryłowa do ruchu 3D |
| **Sterowanie** | amplituda CPG 0,8, 1,5 Hz, przesunięcie; wzmocnienia PID głębokości | dowolny regulator w postaci schematu blokowego |
| **Dane pomiarowe** | pliki CSV dla `calibrate.py`: skok silnika, pompa, przewód, cykle komory, zawór, ogon, ciąg, holowanie i wybieg kadłuba, balast | tablice z `.mat`/`.txt`, import FMU |
| **Uruchomienie** | parametry przez `-override`, modyfikatory w OMEdit | wybór solvera: DASSL, IDA, CVODE, Euler |

## Dane wyjściowe

| WYNIK | ZAWARTOŚĆ | PRZYKŁADOWA WARTOŚĆ |
|---|---|---|
| **Przebiegi czasowe (.mat)** | każda zmienna: p_L, p_R, przepływy, prądy, prędkość silnika, θ, ciąg, prędkość, głębokość, objętość balastu | zawór otwiera się przy ≈51 kPa (HydraulicsStep) |
| **Bilans energii** | strata na każdym komponencie, moc mechaniczna, energia zmagazynowana, błąd bilansu | 1,8 W przy 1 Hz → ≈9,2 h na 16,3 Wh |
| **Pływanie** | prędkość postępowa, ciąg, moc baterii vs częstotliwość | najlepiej 8,5 cm/s przy 0,5 Hz; 16,6 W przy 4 Hz |
| **Regulacja głębokości** | głębokość, położenie tłoka, prąd | zanurzanie 3 cm/s, przeregulowanie 6 %, błąd 5 mm |
| **FMU** | `TailDrive.fmu` (linux64, 1,8 MB) + porównanie z OpenModelica | różnica 0,016 % amplitudy |
| **Raporty kalibracji** | dopasowane parametry z niepewnością 1σ i macierzą korelacji | np. C_d kadłuba 0,402 ± 0,9 % (dane syntetyczne) |



uzwojenie silnika 78,5 % · przewody 9,3 % · łożyska silnika 8,4 % · tarcie pompy 2,7 % · bateria 0,8 % · ogon 0,1 %

Gdzie idzie energia (scenariusz EnergyBudget: 60 s przy 1 Hz, 1,8 W). Przy parametrach zastępczych silnik jest przewymiarowany, więc dominuje jego uzwojenie. W SwimForward tylko ≈0,01 % energii baterii staje się użyteczną pracą napędową. Demo nie zawiera wykresów w repo; generują je skrypty.

## Etapy dema

1–2 Przewód, komora i zawór przelewowy z testami (laminarny, turbulentny vs MSL, obieg zamknięty)

3–4 Bateria, mostek H, silnik, pompa; ogon, CPG i podukład TailDrive; przegląd częstotliwości

5–7 Bilans energii; balast i regulacja głębokości; płetwa i ruch postępowy, pływanie naprzód

8–9 Eksport FMU i pętla w FMPy; wariant `Modelica.Fluid` (te same wyniki z dokładnością 0,5 %, 6× wolniej)

10 Stanowiska kalibracyjne i `calibrate.py` dla 10 podukładów

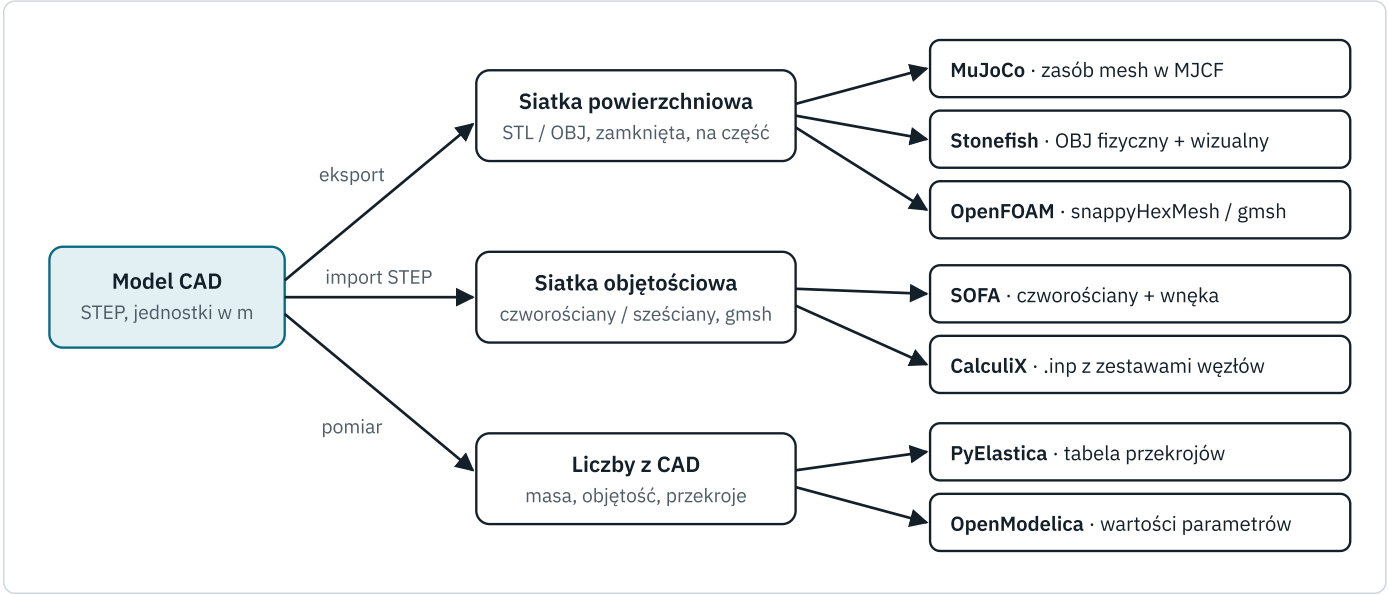
## Ograniczenia

- Ogon ma jeden stopień swobody (brak fali biegnącej); ciąg to zastępczy wzór Lighthilla.
- Ruch postępowy, pionowy i odchylanie nie są sprzężone; opór i masa dodana są stałe.
- Skupiona, nieściśliwa hydraulika bez histerezy; idealna elektryka. FMU działa tylko na systemie, na którym go zbudowano.

# 4 · Od CAD do symulacji oraz ustawianie parametrów

Żadne z sześciu dem nie importuje dziś pliku CAD. MuJoCo, Stonefish i PyElastica budują rybę w kodzie z prostych brył albo z tabeli przekrojów; SOFA i OpenFOAM generują ogon parametrycznie w gmsh; OpenModelica w ogóle nie używa geometrii. Gdy powstanie model CAD prawdziwego robota, każde narzędzie potrzebuje z niego czegoś innego. Ten rozdział pokazuje czego, jak to tam przenieść oraz jak dobiera się i ustawia parametry fizyczne i numeryczne.

## Czego każde narzędzie potrzebuje z modelu CAD




Jeden model STEP daje trzy rodzaje wejścia. Narzędzia brył sztywnych biorą siatki powierzchniowe poszczególnych części; solvery MES biorą siatkę objętościową silikonu z wnękami komór; PyElastica i OpenModelica biorą tylko liczby zmierzone na modelu CAD. Siatkę płynu OpenFOAM buduje się wokół siatki powierzchni ciała.

## Import CAD w poszczególnych narzędziach

| NARZĘDZIE | CZEGO POTRZEBUJE | FORMATY | DROGA Z CAD | NA CO UWAŻAĆ |
|---|---|---|---|---|
| **MuJoCo** | jedna bryła sztywna na część (kadłub, każdy segment ogona, płetwa), z masą i bezwładnością | STL, OBJ jako zasoby `<mesh>` w MJCF; URDF | Podzielić ogon na segmenty w CAD, wyeksportować każdą część do STL/OBJ w metrach, dodać jako geometrię mesh. Masa i bezwładność wynikają z siatki i gęstości albo wpisuje się je z właściwości masowych CAD. URDF wyeksportowany z CAD można wczytać i zapisać jako MJCF. | Kolizje używają otoczki wypukłej siatki. Model płynu widzi tylko równoważną elipsoidę dla każdej geometrii, a nie kształt siatki. Wypór nadal trzeba dodać samemu (jak w demie). |
| **Stonefish** | zamknięta siatka fizyczna o małej liczbie trójkątów i gładka siatka wizualna dla każdego ogniwa | OBJ (użyty tutaj), wskazywany w XML `.scn` | Wyeksportować każde ogniwo do OBJ, kopię fizyczną zdecymować do kilkuset trójkątów (demo: 320), wizualną zostawić dokładną. `tools/make_meshes.py` pokazuje cały proces z trimesh + manifold3d: test szczelności, objętości, bilans masy, dobór balastu, wypełnienie XML z szablonów. | Wypór, opór i masa dodana liczone są na każdym trójkącie, więc siatka musi być zamknięta, z normalnymi na zewnątrz. Więcej trójkątów spowalnia każdy krok. Materiały można definiować tylko w głównym pliku scenariusza. |
| **PyElastica** | bez siatki: linia środkowa i jeden przekrój na element | brak (liczby w `fishrod/config.py`) | Pociąć ciało z CAD w stacjach wzdłuż długości (np. `trimesh.section`), odczytać półszerokość a(s) i półwysokość b(s) albo od razu policzyć pole i momenty bezwładności przekroju, i wypełnić tabelę profilu. | Pręt nie ma wnęk. Komory wchodzą tylko przez A_r (zgięcie na ciśnienie) i C_h (podatność), które lepiej zmierzyć niż liczyć. |
| **SOFA** | czworościenna siatka objętościowa silikonu i zamknięta powierzchnia z trójkątów dla każdej wnęki komory | `.msh` (Gmsh), `.vtk` (legacy 4.2), powierzchnie STL/OBJ | W gmsh zastąpić parametryczną geometrię z `fishsofa/mesh_gen.py` (już zbudowaną jądrem OCC) wywołaniem `gmsh.model.occ.importShapes("tail.step")`. Zdefiniować grupy fizyczne dla utwierdzonej nasady i ścian wnęk, potem siatkować. | Przez każdą cienką ściankę co najmniej 2–3 elementy: przy 1–2 liniowych czworościanach ciśnienie wychodzi o 15–25 % za wysokie. VTK 5.1 wywraca `MeshVTKLoader`. Powierzchnia każdej wnęki musi być zamknięta. |

| NARZĘDZIE | CZEGO POTRZEBUJE | FORMATY | DROGA Z CAD | NA CO UWAŻAĆ |
|---|---|---|---|---|
| **OpenFOAM + CalculiX** | płyn: siatka wody wokół ciała z nazwanymi fragmentami brzegu; ciało stałe: siatka MES ogona z zestawami węzłów | płyn: STL dla snappyHexMesh albo `.msh` przez `gmshToFoam` (użyte tutaj); ciało stałe: `.inp` CalculiX | Płyn: wyeksportować powierzchnię ciała do STL i siatkować blockMesh + snappyHexMesh (3D) albo zaimportować STEP do gmsh jak w `tools/make_geometry.py`. Ciało stałe: siatkować w gmsh lub FreeCAD FEM i zapisać `.inp` z zestawami dla utwierdzonej nasady, powierzchni zwilżonej (sprzężenie) i ścian komór (obciążenie ciśnieniem). | Nazwy fragmentów brzegu i zestawów muszą zgadzać się z `preciceDict` i `config.yml`. Siatka płynu odkształca się z ogonem, co ogranicza amplitudę końca do ≈15 % L. Przypadek 2D wymaga wyciągnięcia na grubość jednej komórki. |
| **OpenModelica** | bez geometrii; tylko wartości skupione | brak dla fizyki; MultiBody może pokazać kształty STL jako wizualizację | Odczytać liczby z modelu CAD: masy, bezwładności, środek wyporu, objętość komory i jej zmianę na radian zgięcia (D_tail = dV/dθ), długości i średnice przewodów, średnicę i skok strzykawki. | Krzywa p–V komory zależy od silikonu, nie tylko od kształtu: wziąć ją z SOFA albo z próby strzykawką (model już czyta ją z CSV). |

## Jak dobiera się parametry

Każda wartość w demach jest zastępcza. Prawdziwe wartości pochodzą z pięciu źródeł, w tej kolejności:

1. **Karty katalogowe:** R, L, k i bezwładność wirnika silnika, wydajność pompy na obrót, napięcie i pojemność baterii, częstotliwość i szum czujników.
2. **Właściwości masowe z CAD:** masy, objętości, środki ciężkości i wyporu, bezwładności, objętość komory.
3. **Badania materiału:** gęstość i moduł Younga silikonu (próba rozciągania lub wciskania odlanej próbki).
4. **Identyfikacja na stanowiskach, zgodnie z przepływem energii:** silnik → pompa → przewód → komora → zawór → statyka ogona → dynamika ogona → ciąg → kadłub → balast. Każdy krok ustala kilka parametrów, gdy poprzednie są już znane. To 11-krokowa procedura z dema OpenModelica (`calibrate.py`); te same pomiary służą pozostałym narzędziom.
5. **Dokładne modele jako odniesienie dla prostych:** krzywa p–V z SOFA wypełnia tablicę komory w OpenModelica; amplituda końca, masa dodana i ciąg z OpenFOAM sprawdzają współczynniki wody w MuJoCo, Stonefish i PyElastica.

Parametry numeryczne (krok czasu, rozmiar siatki, liczba elementów) nie są fizyką. Dobiera się je z warunku stabilności i sprawdza testem zbieżności, nigdy nie stroi pod wynik. Demo Stonefish mówi to wprost: gdy pierwsza wersja płynęła 0,3 cm/s, zdiagnozowano przyczynę zamiast stroić współczynniki, aż ryba ruszy.

| NARZĘDZIE | KROK CZASU | SIATKA LUB DYSKRETYZACJA |
|---|---|---|
| **MuJoCo** | 2 ms, `implicitfast`; najsztywniejszy mod ω_n·dt ≈ 0,05 (warunek: < 0,5) | 5 segmentów ogona (z konstrukcji, nie z testu zbieżności) |
| **Stonefish** | 0,5 ms: jawne sprężyny przegubów wymagają ω·dt < 2, a 2 ms eksplodowało; 4000 Hz daje tę samą prędkość | siatka fizyczna 320 trójkątów na bryłę |
| **PyElastica** | dt < min(Δl/c, Δl²/(r·c)) ≈ 19 µs, liczony automatycznie | n = 50 elementów: błąd ≈1 % względem teorii belek |
| **SOFA** | 2 ms w powietrzu; 0,5 ms w wodzie, z warunku jawnego oporu c·dt/m < 0,5 | coarse/medium/fine; zgrubna zawyża ciśnienie o 20–27 % względem dokładnej |
| **OpenFOAM FSI** | 2,5 ms, ten sam w OpenFOAM, CalculiX i oknie preCICE | trzy siatki: 1,0 % zmiany oporu między dwiema najgęstszymi; domena 1,5× zmienia opór o 3,8 % |
| **OpenModelica** | zmienny krok (DASSL), tolerancja $10^{-6}$ | brak; atrybuty `nominal` skalują zmienne |

## Gdzie ustawia się parametry i który pomiar je identyfikuje

| NARZĘDZIE | GDZIE SĄ | JAK JE ZMIENIĆ | LICZONE AUTOMATYCZNIE | IDENTYFIKOWANE PRZEZ (PLAN Z README) |
|---|---|---|---|---|
| **MuJoCo** | dataclass `FishConfig`, `fishsim/config.py` | edycja pliku albo `FishConfig(tail_freq=1.5)` w Pythonie; model MJCF generuje się na nowo | masa kadłuba (neutralna przy połowie pęcherza), położenie środka wyporu wzdłuż (trym), sztywność przegubów pasywnych (z rezonansu 3 Hz) | ogon na stole (ciśnienie vs kąt → A·r, k przegubów, C_h); ogon w wodzie (rezonans, tłumienie); ciąg na uwięzi; nagranie w basenie (prędkość → C_blunt, C_slender; skręt → C_Kutta); ważenie pęcherza pod wodą |
| **Stonefish** | `config/default.json` + łatki scenariuszy `config/sN_*.json` | edycja JSON albo `--set rhythm.freq=1.5`; bez rekompilacji | masa i położenie stalowego balastu, neutralna objętość VBS, objętości siatek (`make_meshes.py`) | procedura MuJoCo bez zmian, plus tłumienie kołysania i promień skrętu |
| **PyElastica** | dataclassy w `fishrod/config.py` (Rod, Fish, Hydraulics, Water, Buoyancy, etapy) | edycja albo `HydraulicsConfig(tail_freq=1.5)` w skrypcie | stabilny krok czasu, położenie pęcherza, macierze sztywności przekrojów | zgięcie vs ciśnienie i objętość (A_r, C_h, E); częstotliwość własna i tłumienie w powietrzu i wodzie; ciąg na uwięzi i prędkość w basenie (C_n, C_t, współczynnik reakcyjny); skok głębokości (pompa pęcherza, PID) |
| **SOFA** | `TailConfig` w `fishsofa/config.py`; zmienne środowiskowe `FISHSOFA_*` | edycja, np. `young_modulus`, `tail_freq`; siatka generuje się na nowo i jest buforowana według skrótu geometrii | siatki, objętości wnęk, masa wody w komorach | strzykawka do jednej komory co 5 ml z manometrem i zdjęciem z góry: dopasować E do krzywej p–V, potem współczynnik kręgosłupa i sztywność włókien do krzywej kąt–objętość; ciąg na wadze w wannie |

| NARZĘDZIE | GDZIE SĄ | JAK JE ZMIENIĆ | LICZONE AUTOMATYCZNIE | IDENTYFIKOWANE PRZEZ (PLAN Z README) |
|---|---|---|---|---|
| **OpenFOAM FSI** | `tools/params.py` (geometria, materiały); `system/caseParams`; słowniki; `fsi.inp`; `precice-config.xml` | `new-fsi-case.sh --u 0.1 --p0 15000 --freq 1.5 --coupling implicit` tworzy nowy przypadek | siatki (gmsh), tablice ciśnienia napędu (`make_actuation.py`) | głowa na jednoosiowym czujniku siły w zbiorniku, kamera z góry: średnia siła osiowa, amplituda końca na sucho i w wodzie, prędkość napływu przy zerowej średniej sile → E i tłumienie |
| **OpenModelica** | deklaracje `parameter` w plikach `.mo` | okno parametrów OMEdit, modyfikatory typu `cpg(f=1.5)` albo `-override cpg.f=1.5` przy uruchomieniu (OMPython w skryptach) | krzywa p–V komory z pliku CSV, składniki bilansu energii | `calibrate.py` z 10 modelami stanowisk (silnik, pompa, przewód, komora, zawór, ogon statycznie i dynamicznie, ciąg, holowanie i wybieg kadłuba, balast): najmniejsze kwadraty z niepewnością 1σ i macierzą korelacji |

# 5 · Porównanie

## Macierz możliwości

● zamodelowane w demie ◐ uproszczone lub częściowe ○ niezamodelowane

| MOŻLIWOŚĆ | ● MUJOCO | ● STONEFISH | ● PYELASTICA | ● SOFA | ● OPENFOAM FSI | ● OPENMODELICA |
|---|---|---|---|---|---|---|
| **Swobodne pływanie całej ryby** | ● | ● | ● | ○ utwierdzony | ○ głowa nieruchoma | ◐ ruch postępowy + pionowy |
| **Odkształcenie miękkiego ogona** | ◐ 5 segmentów | ◐ 5 segmentów | ◐ ciągły pręt | ● MES 3D | ● MES 2D | ○ 1 st. swobody |
| **Ciśnienie w komorze z konstrukcji** | ◐ stałe C_h | ◐ stałe C_h | ◐ stałe C_h | ● | ◐ ciśnienie jest wejściem | ◐ tablica p–V |
| **Przepływ wody, ślad, wiry** | ○ | ○ | ○ | ○ | ● | ○ |
| **Siły hydrodynamiczne** | ◐ model elipsoid | ◐ na ściankach siatki | ◐ Morison + Lighthill | ◐ tylko opór | ● Navier–Stokes | ◐ Lighthill + C_d |
| **Masa dodana** | ◐ diagonalna | ◐ uśredniona po osiach | ◐ krawędź spływu | ○ | ● z przepływu | ◐ stała |
| **Dynamika pompy i zaworu** | ◐ | ◐ | ◐ | ◐ | ○ | ● |
| **Elektryka i bilans energii** | ○ | ○ | ○ | ○ | ○ | ● |
| **Wyporność i regulacja głębokości** | ● | ● | ● | ○ | ○ | ◐ pion 1D |
| **Zaszumione czujniki** | ○ możliwe | ● | ○ | ○ | ○ | ○ |
| **Prądy i środowisko** | ○ | ● | ○ | ○ | ◐ napływ | ○ |
| **Interaktywny podgląd 3D** | ● | ● | ○ GIF | ● | ◐ ParaView, po obliczeniach | ○ wykresy |
| **Czas rzeczywisty lub szybciej** | ● | ● 5–8× | ◐ ≈1,5× wolniej | ○ 100–800× wolniej | ○ godziny | ● milisekundy |
| **Identyfikacja parametrów** | ○ tylko plan | ○ | ○ tylko plan | ○ tylko plan | ○ | ● calibrate.py |
| **Kosymulacja / eksport** | ○ | ◐ ROS 2 możliwy | ○ | ○ | ● preCICE | ● FMU 2.0 |

## Wejścia w skrócie

| WEJŚCIE | ●MUJOCO | ●STONEFISH | ●PYELASTICA | ●SOFA | ●OPENFOAM FSI | ●OPENMODELICA |
|---|---|---|---|---|---|---|
| **Geometria** | MJCF generowany w Pythonie | siatki OBJ + XML `.scn` | tabela stacji w Pythonie | gmsh → VTK + OBJ | gmsh → siatka OpenFOAM + siatka CalculiX | brak (parametry skupione) |
| **Materiał** | gęstości, sztywność przegubów | gęstości, sztywność przegubów | E, ν, ρ dla elementu | E, ν, ρ, włókna, kręgosłup | E, ν, ρ | k, J, c ogona |
| **Hydraulika** | skupiona (A, r, C_h) | skupiona (A, r, C_h) | skupiona (A_r, C_h) | objętość na komorę | ciśnienie na komorę | parametry komponentów + tablica p–V |
| **Woda** | `fluidcoef` | z siatki + nachylenie siły nośnej | C_n, C_t, wybór modelu | C_n, C_t | ρ, ν, napływ U | C_d, C_T, masa dodana |
| **Format konfiguracji** | dataclass w Pythonie | JSON + XML | dataclass w Pythonie | dataclass w Pythonie + zmienne środowiskowe | słowniki, `.inp`, XML | parametry Modelica |
| **Sterowanie na żywo** | klawiatura | klawiatura | brak | GUI | brak | brak |

## Wyjścia w skrócie

| WYJŚCIE | MUJOCO | STONEFISH | PYELASTICA | SOFA | OPENFOAM FSI | OPENMODELICA |
|---|---|---|---|---|---|---|
| **Główna wielkość** | prędkość, trajektoria, głębokość | prędkość, trajektoria, odczyty czujników | prędkość, kształt ciała, ciąg | ciśnienie, zgięcie, ciąg na uwięzi | pole przepływu, siły, ślad | prądy, ciśnienia, energia |
| **Format danych** | CSV | CSV 100 Hz + JSON | CSV | CSV + NPZ | pola OpenFOAM, `.frd`, logi | `.mat`, CSV |
| **Wizualizacja** | podgląd, PNG | GUI 3D, PNG | PNG, GIF | GUI, MP4 | ParaView, PNG | wykresy PNG |
| **Odczyty na ekranie** | nakładka: stan, napęd, siła wody na każde ciało; strzałki sił | panel: stan, napęd, opór części, siła nośna płetwy; linie sił biblioteki | panel animacji: prędkość, Δp, kąty, opór, siła reaktywna; strzałki | tekst: hydraulika, ogon, siła na odcinki; strzałki; skóra barwiona przemieszczeniem, obciążeniem lub naprężeniem von Misesa | tekst w ParaView: siła na ciało i ogon, moment na ogonie; strzałki | OMEdit: wykres dowolnej zmiennej po symulacji |

## Ta sama ryba, różne prędkości: dlaczego

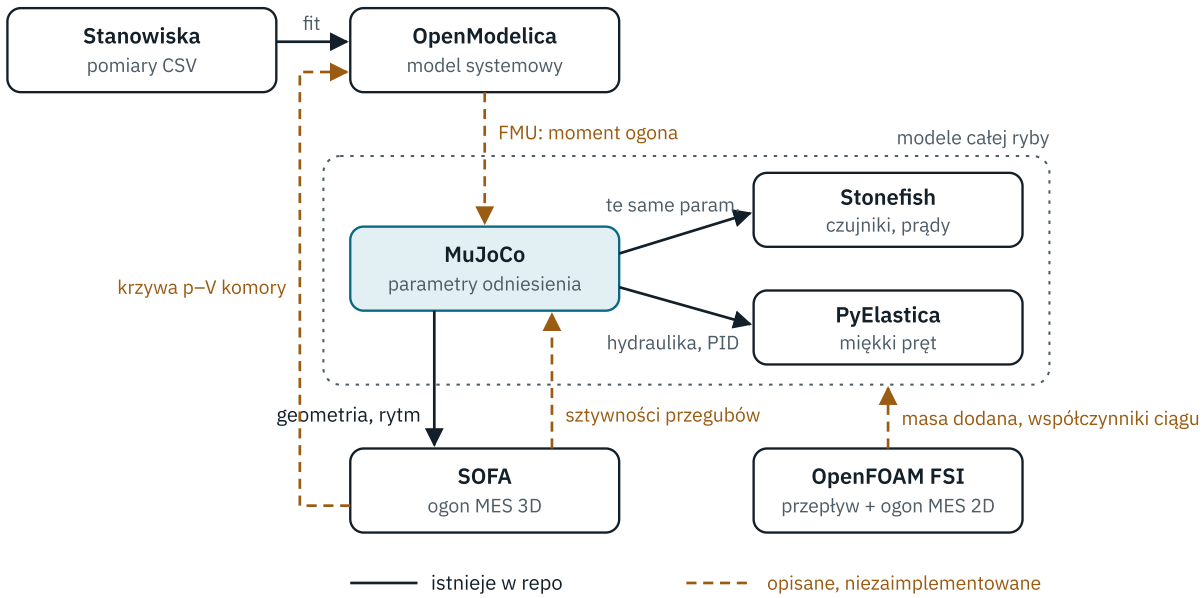
Przy około 2 Hz trzy dema pływania swobodnego dają prędkości różniące się 2×. Różnica wynika z tego, skąd każde narzędzie bierze ciąg.

| NARZĘDZIE | PRĘDKOŚĆ PRZY OK. 2 HZ | GŁÓWNE ŹRÓDŁO CIĄGU W MODELU |
|---|---|---|
| **MuJoCo** | 22 cm/s | siła nośna Kutty na elipsoidach z wbudowanego modelu |
| **Stonefish** | 34 cm/s | siła nośna płaskiej płetwy dodana w C++ |
| **PyElastica** | 44 cm/s | siła reakcyjna Lighthilla na krawędzi spływu |
| **OpenModelica** | 7,3 cm/s | sztywna płetwa Lighthilla na ogonie o 1 st. swobody (tylko 3,4° amplitudy przy 2 Hz) |
| **SOFA** | na uwięzi | opór na trójkątach skóry: 63 mN przy 2 Hz |
| **OpenFOAM FSI** | na uwięzi | rozwiązany przepływ: opór wypadkowy przy tym kształcie ogona |

## Koszt użycia

| NARZĘDZIE | INSTALACJA | TYPOWE URUCHOMIENIE | POTRZEBNE UMIEJĘTNOŚCI |
|---|---|---|---|
| **MuJoCo** | pip, kilka minut | sekundy do minut | Python |
| **Stonefish** | budowa ze źródeł, GPU do GUI | 1–2 min dla wszystkich scenariuszy | C++, XML |
| **PyElastica** | pip, kilka minut | 9–70 s na etap | Python, mechanika prętów |
| **SOFA** | skrypt, ≈2 min + kompilator dla CHOLMOD | 10 min do 1 h na etap | Python, MES, siatkowanie |
| **OpenFOAM FSI** | ≈1,5 h, trzy programy i adaptery ze źródeł | 7–102 min na przypadek | CFD, MES, sprzęganie |
| **OpenModelica** | repozytorium apt + venv | poniżej sekundy | Modelica |

## Jak narzędzia się uzupełniają




MuJoCo jest punktem odniesienia: Stonefish używa tej samej ryby i parametrów, PyElastica przenosi jego hydraulikę i PID głębokości, a SOFA kopiuje geometrię ogona i rytm. Linie przerywane to planowane kolejne kroki, każdy opisany w jakimś README: krzywe p–V z SOFA do tablicy komory w OpenModelica, FMU z OpenModelica napędzające przegub MuJoCo, sztywności z SOFA/PyElastica eksportowane do przegubów MuJoCo oraz wyniki OpenFOAM do kalibracji tańszych modeli wody.

# 6 · Słownik

| POLSKI | ANGIELSKI | ZNACZENIE W TYM RAPORCIE |
|---|---|---|
| **FMU** | Functional Mock-up Unit | spakowany model, który mogą uruchamiać inne narzędzia (eksport z OpenModelica) |
| **generator rytmu (CPG)** | central pattern generator | regulator wytwarzający okresowe zadanie objętości dla ogona |
| **identyfikacja parametrów** | identification | dopasowanie parametrów modelu do pomiarów ze stanowisk (OpenModelica `calibrate.py`) |
| **interakcja płyn–struktura** | FSI | sprzężona symulacja przepływu wody i odkształcającego się ciała (OpenFOAM + CalculiX) |
| **IQN-ILS** | IQN-ILS (quasi-Newton) | metoda, którą preCICE doprowadza sprzężenie niejawne do zbieżności |
| **krzywa ciśnienie–objętość** | p–V curve | ciśnienie w komorze w funkcji wpompowanej objętości wody |
| **liczba Strouhala** | Strouhal number | St = f·A/U; zwierzęta pływają wydajnie przy St ≈ 0,2–0,4, większość ryb 0,25–0,35 (Taylor et al. 2003; Triantafyllou et al. 1993) |
| **masa dodana** | added mass | woda, którą trzeba przyspieszyć razem z ciałem; w demie FSI obniża rezonans ogona z 3,39 Hz do ≈1 Hz |
| **MES korotacyjna** | corotational FEM | sprężystość liniowa poprawnie obsługująca duże obroty (SOFA) |
| **model pseudo-sztywny** | PRBM | miękki ogon przybliżony sztywnymi segmentami i sprężynami (MuJoCo, Stonefish) |
| **na uwięzi** | tethered | ciało trzymane w miejscu podczas pomiaru ciągu (SOFA, OpenFOAM) |
| **pręt Cosseratów** | Cosserat rod | kontinuum 1D, które się zgina, skręca, ścina i rozciąga (PyElastica) |
| **siatka szczelna (zamknięta)** | watertight mesh | zamknięta powierzchnia bez dziur; potrzebna do wyporu w Stonefish i wnęk w SOFA |
| **siła reakcyjna Lighthilla** | Lighthill reactive force | ciąg od wody odpychanej na boki przez krawędź spływu |
| **STEP** | STEP | neutralny format wymiany CAD (ISO 10303); gmsh importuje go bezpośrednio |
| **układ zmiennej pływalności** | VBS | zbiornik, którego objętość wody ustala głębokość (Stonefish) |
| **zawór przelewowy** | relief valve | zawór otwierający się przy \|Δp\| = 50 kPa, chroni komory |

* Licencje nie są podane w repozytorium. Wymienione licencje dotyczą samych narzędzi, według ich publicznych dystrybucji.